

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT2-Constraint-Based Problem Solving

Register Number	192511025
Name	S.Swedha

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 20%

QUESTIONS:3

Total Marks:30

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information. The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I _____ S.Swedha/192511025 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5

Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

Answers:

1) Doctor Shift Scheduling using Backtracking Search

a. Problem Statement

We need to assign 3 doctors D1, D2, D3 to 3 shifts Morning, Afternoon, Night satisfying these constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

This is a Constraint Satisfaction Problem (CSP) and can be solved using Backtracking Search.

b. Variables and Domains

Variables: D1, D2, D3

Domain for each variable: {Morning, Afternoon, Night}

c. Constraints

C1: $D1 \neq \text{Night}$

C2: $\text{Position}(D2) < \text{Position}(D3)$, taking Morning=1, Afternoon=2, Night=3

C3: All-Different — no two doctors can get the same shift

C4: $D3 \neq \text{Morning}$

C5: Every doctor gets exactly one shift

d. Backtracking Search Procedure

Assign variables in order $D1 \rightarrow D2 \rightarrow D3$. Try domain values in order Morning, Afternoon, Night. After every assignment, check it against the constraints that apply so far. If a constraint fails, discard that value and backtrack; if it holds, move to the next variable.

e. Step-by-Step Trace

Step 1: Assign $D1 = \text{Morning}$

Check C1 ($D1 \neq \text{Night}$) $\rightarrow$ satisfied. Continue.

Step 2: Assign D2 = Afternoon

No constraint applies yet (C2 needs D3 too). Continue.

Step 3: Assign D3 = Night

Check C4 ($D3 \neq \text{Morning}$) $\rightarrow$ satisfied (it's Night)

Check C2 ($\text{Afternoon} < \text{Night}$) $\rightarrow$ satisfied

Check C3 (all different) $\rightarrow$ satisfied

$\rightarrow$ All constraints satisfied. **Solution 1 found: D1=Morning, D2=Afternoon, D3=Night**

Backtrack to D2, try next value.

Step 4: Assign D2 = Night

No conflict yet. Continue.

Step 5: Assign D3 = Afternoon

Check C2 ($\text{Night} < \text{Afternoon?}$) $\rightarrow$ FAILS, since Night comes after Afternoon.

Backtrack.

No more values left for D2. Backtrack to D1.

Step 6: Assign D1 = Afternoon

Check C1 ($D1 \neq \text{Night}$) $\rightarrow$ satisfied. Continue.

Step 7: Assign D2 = Morning

No conflict yet. Continue.

Step 8: Assign D3 = Night

Check C4 ($D3 \neq \text{Morning}$) $\rightarrow$ satisfied

Check C2 ($\text{Morning} < \text{Night}$) $\rightarrow$ satisfied

Check C3 $\rightarrow$ satisfied

$\rightarrow$ All constraints satisfied. **Solution 2 found: D1=Afternoon, D2=Morning, D3=Night**

Backtrack to D2, try next value.

Step 9: Assign D2 = Night

No conflict yet. Continue.

Step 10: Assign D3 = Morning

Check C4 ($D3 \neq \text{Morning}$) $\rightarrow$ FAILS immediately.

Backtrack. No values left for D2 either. Backtrack to D1.

Step 11: Assign D1 = Night

Check C1 ($D1 \neq \text{Night}$) $\rightarrow$ FAILS immediately.

This entire branch is pruned — D2 and D3 are never even attempted, since D1 alone already breaks a constraint.

f. Observations

- D1 = Night is eliminated at the very first level, which prunes 2 whole sub-branches instantly.
- The ordering constraint (C2) removes any case where D2 is scheduled after D3.
- The constraint C4 removes any case where D3 = Morning, no matter what D1 and D2 are.
- Out of 6 total possible ways to assign 3 shifts to 3 doctors, only 2 satisfy every constraint.

g. Final Valid Schedules

Doctor	Solution 1	Solution 2
D1	Morning	Afternoon
D2	Afternoon	Morning
D3	Night	Night

If only one schedule is required, Solution 1 is usually taken since it's the first one the algorithm reaches:

D1 = Morning, D2 = Afternoon, D3 = Night

2) Problem Statement

A robot must move from Start (S) to Goal (G) in a 5×5 grid, where certain cells are blocked by obstacles.

Constraints:

- Allowed moves: Up, Down, Left, Right
- Cost of each move = 1
- Robot cannot pass through obstacle cells
- Manhattan Distance is used as the heuristic $h(n)$

Since the problem requires both a cost function $g(n)$ and a heuristic $h(n)$ combined through a priority queue, this is solved using A* Search, where $f(n) = g(n) + h(n)$.

Grid Representation

Rows and columns are numbered 0 to 4. S is at (0,0), G is at (4,4).

(0,0)S (0,1) (0,2) (0,3) (0,4)

(1,0) (1,1)X (1,2)X (1,3) (1,4)

(2,0) (2,1) (2,2) (2,3)X (2,4)

(3,0) (3,1)X (3,2) (3,3) (3,4)

(4,0) (4,1) (4,2) (4,3) (4,4)G

Obstacles (X): (1,1), (1,2), (2,3), (3,1)

Heuristic Function

Manhattan Distance: $h(n) = |\text{row}(n) - \text{row}(G)| + |\text{col}(n) - \text{col}(G)|$

Step-by-Step Node Expansion

Start node (0,0): $g=0$, $h=8$, $f=8$

Step 1: Expand (0,0)

Neighbors generated: (1,0) and (0,1) — Left and Up are outside the grid.

(1,0): $g=1$, $h=7$, $f=8$

(0,1): $g=1$, $h=7$, $f=8$

Priority Queue: (1,0) $f=8$, (0,1) $f=8$

Step 2: Expand (1,0) [lower h , tie-break]

Neighbors: Up=(0,0) already visited, Right=(1,1) is an obstacle — blocked, Down=(2,0)

(2,0): $g=2$, $h=6$, $f=8$

Priority Queue: (0,1) $f=8$, (2,0) $f=8$

Step 3: Expand (2,0) [$h=6$ is lower than (0,1)'s $h=7$]

Neighbors: Up=(1,0) visited, Down=(3,0), Right=(2,1)

(3,0): $g=3$, $h=5$, $f=8$

(2,1): $g=3$, $h=5$, $f=8$

Priority Queue: (0,1) $f=8$, (3,0) $f=8$, (2,1) $f=8$

Step 4: Expand (3,0) [h=5, generated before (2,1)]

Neighbors: Up=(2,0) visited, Right=(3,1) is an obstacle — blocked, Down=(4,0)

(4,0): g=4, h=4, f=8

Priority Queue: (0,1) f=8, (2,1) f=8, (4,0) f=8

Step 5: Expand (4,0) [h=4, lowest]

Neighbors: Up= (3,0) visited, Right= (4,1)

(4,1): g=5, h=3, f=8

Priority Queue: (0,1) f=8, (2,1) f=8, (4,1) f=8

Step 6: Expand (4,1) [h=3, lowest]

Neighbors: Up= (3,1) is an obstacle — blocked, Left= (4,0) visited, Right=(4,2)

(4,2): g=6, h=2, f=8

Priority Queue: (0,1) f=8, (2,1) f=8, (4,2) f=8

Step 7: Expand (4,2) [h=2, lowest]

Neighbors: Up=(3,2), Left=(4,1) visited, Right=(4,3)

(3,2): g=7, h=3, f=10

(4,3): g=7, h=1, f=8

Priority Queue: (0,1) f=8, (2,1) f=8, (3,2) f=10, (4,3) f=8

Step 8: Expand (4,3) [h=1, lowest]

Neighbors: Up=(3,3), Left=(4,2) visited, Right=(4,4) = Goal

(3,3): g=8, h=2, f=10

(4,4): g=8, h=0, f=8

Priority Queue: (0,1) f=8, (2,1) f=8, (3,2) f=10, (3,3) f=10, (4,4) f=8

Step 9: Expand (4,4)

(4,4) has the lowest f value (f=8, h=0) in the queue, so it is selected next. Since it is the Goal node, the search terminates here. All other nodes still in the queue are never expanded.

Optimal Path

Tracing back through parent pointers from Goal to Start:

(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

In terms of moves: Down, Down, Down, Down, Right, Right, Right, Right

Optimal Cost

Total number of moves = 8

Total path cost = 8 (since each move costs 1)

This matches the Manhattan Distance between $S(0,0)$ and $G(4,4)$, which is $|0-4| + |0-4| = 8$. Since the actual cost equals the heuristic's initial estimate, this confirms the path found is optimal — no shorter path is possible, and the obstacles did not force any detour.

Observations

- Nodes (1,1), (1,2), and (2,3) were never expanded because moving into them was blocked immediately by the obstacle check.
- At several steps, multiple nodes had the same $f(n)$ value; the heuristic (lower h) was used to decide which one to expand first, showing how the heuristic actively guides the search toward the Goal rather than expanding blindly in all directions like plain BFS would.
- Nodes (3,2) and (3,3), though valid, ended up with higher f -values (10) and were never expanded, since the goal was reached before the search needed to consider them — this demonstrates the pruning efficiency A^* gains from using a heuristic.

Algorithm: A^* Search

1. Initialize:

- Create a priority queue (open list) and add the Start node
- $g(\text{Start}) = 0$
- $h(\text{Start}) = \text{Manhattan Distance}(\text{Start}, \text{Goal})$
- $f(\text{Start}) = g(\text{Start}) + h(\text{Start})$
- Create an empty closed list (visited nodes)

2. Loop while the priority queue is not empty:

- a. Remove the node with the lowest $f(n)$ from the priority queue
(if tie, choose the node with lower $h(n)$)

- b. If this node is the Goal, stop — path found. Go to Step 4.
- c. Add this node to the closed list
- d. For each neighbor (Up, Down, Left, Right) of the current node:
 - i. If neighbor is outside the grid → skip
 - ii. If neighbor is an obstacle → skip
 - iii. If neighbor is already in the closed list → skip
 - iv. Calculate tentative $g(\text{neighbor}) = g(\text{current}) + 1$
 - v. If neighbor is not in the priority queue, or the new g is lower than its previous g :
 - Update $g(\text{neighbor})$
 - Calculate $h(\text{neighbor}) = \text{Manhattan Distance}(\text{neighbor}, \text{Goal})$
 - Calculate $f(\text{neighbor}) = g(\text{neighbor}) + h(\text{neighbor})$
 - Set $\text{parent}(\text{neighbor}) = \text{current}$
 - Add/update neighbor in the priority queue
- 3. If the priority queue becomes empty without reaching the Goal → no path exists.
- 4. Reconstruct path:
 - Starting from Goal, follow parent pointers back to Start

- Reverse the sequence to get the path from Start to Goal

3) Problem Statement

An autonomous rescue robot operates in a disaster-affected building represented as a grid of rooms. It starts at position S and must reach a survivor at position G. The environment is partially observable — the robot can only sense adjacent cells — and it must navigate safely while minimizing total movement cost, updating its knowledge of the environment as it moves (an online search scenario).

(a): Analysis of Constraints and Their Effect on Decision-Making

Movement restricted to Up, Down, Left, Right

This defines the robot's action set. At every cell, the robot has at most 4 possible next moves, which keeps the branching factor small and decision-making manageable, but also means the robot cannot take shortcuts (like diagonal moves) even if that would reduce cost.

Some cells are blocked (obstacles)

The robot cannot know in advance which cells are blocked beyond what it has already sensed. This forces the robot to treat some paths as "unknown" until it physically reaches and senses them, meaning a path that looks promising may turn out to be a dead end only after the robot arrives there.

Limited sensing (only adjacent cells observable)

This is the defining constraint that makes the problem partially observable. The robot cannot plan a complete path from S to G in advance the way a fully-informed search algorithm could. It must make decisions incrementally, sensing, then deciding, then moving, and only expands its understanding of the map step-by-step.

Movement cost = 1, risky zone cost = +2

This introduces a non-uniform cost structure. The robot cannot simply choose the path with the fewest steps; it must weigh whether a shorter-looking path through a risky zone is actually more expensive than a longer path through safe cells. This requires the robot to track and compare cumulative path cost, not just distance.

Must minimize total cost while ensuring safe navigation

This defines the performance measure the robot is optimizing. It creates a trade-off: the robot must balance cost-efficiency against safety, meaning the "cheapest" move is not automatically the "best" move if it leads into greater risk or an unrecoverable dead end.

Must update knowledge dynamically (online search)

This means the robot cannot rely on a static, pre-computed plan. Every time it senses new information (a newly discovered obstacle, a risky zone), it must revise its internal map and potentially reconsider its next action, rather than blindly following an initial plan.

(b): Step-by-Step Online Search Trace

The robot starts at (0,0), $g = 0$. At each cell it senses adjacent cells, updates its known map, and picks the lowest-cost known move.

Step 1: At (0,0). Senses Right (0,1) = free, Down (1,0) = free. Both cost 1 — picks Right (arbitrary tie-break, heading toward Goal's column). Moves to (0,1). $g = 1$.

Step 2: At (0,1). Senses Right (0,2) = free, Down (1,1) = obstacle. Only Right is viable. Moves to (0,2). $g = 2$.

Step 3: At (0,2). Senses Right (0,3) = obstacle, Down (1,2) = free. Only Down is viable. Moves to (1,2). $g = 3$.

Step 4: At (1,2). Senses Down (2,2) = obstacle, Right (1,3) = obstacle, Left (1,1) = obstacle (already known). **Dead end** — every neighbor except the cell it came from is blocked. The robot's knowledge map is updated to mark this branch as fully explored/unusable.

Step 5 (Backtrack): Robot retreats: $(1,2) \rightarrow (0,2)$. Cost of this move = 1. $g = 4$. At (0,2), all neighbors are now either obstacles or already explored — continue backtracking.

Step 6 (Backtrack): $(0,2) \rightarrow (0,1)$. $g = 5$. At (0,1), Right leads to the dead-end branch (already explored) and Down is an obstacle — continue backtracking.

Step 7 (Backtrack): $(0,1) \rightarrow (0,0)$. $g = 6$. At (0,0), the only remaining unexplored neighbor is Down (1,0).

Step 8: Moves to (1,0). $g = 7$. Senses Down (2,0) = risky, Right (1,1) = obstacle (known). Only Down (risky) is viable.

Step 9: Moves into risky zone (2,0). Cost of this move = $1 + 2 = 3$. $g = 10$. Senses Down (3,0) = free, Right (2,1) = obstacle. Only Down viable.

Step 10: Moves to (3,0). $g = 11$. Senses Right (3,1) = free. Moves to (3,1). $g = 12$.

Step 11: At (3,1). Senses Right (3,2) = free. Moves to (3,2). $g = 13$.

Step 12: At (3,2). Senses Right (3,3) = free. Moves to (3,3). $g = 14$.

Step 13: At (3,3). Senses Right (3,4) = Goal! Moves to (3,4) = G. $g = 15$. **Goal reached.**

Final Path and Cost

Successful route actually taken by the robot (including the dead-end detour and backtracking):

$(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2)$ [dead end, sensed] $\rightarrow (0,2) \rightarrow (0,1) \rightarrow (0,0) \rightarrow (1,0) \rightarrow (2,0)$ [risky] $\rightarrow (3,0) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4)$

Total moves = 13

Total cost incurred = 15 (12 normal moves $\times$ 1, plus 1 risky move $\times$ 3)

The **underlying optimal path**, if the robot had known the full map in advance, would simply have been:

$(0,0) \rightarrow (1,0) \rightarrow (2,0)$ [risky] $\rightarrow (3,0) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4)$
= 7 moves, cost = 9 ($6 \times 1 + 1 \times 3$)

The difference (15 vs. 9) is the real cost of partial observability — the robot paid an extra 6 units of cost purely for exploring and backtracking out of the dead end, since it had no way of knowing that branch was blocked until it physically sensed it.

(c): AI Concepts Demonstrated

Intelligent Agent (PEAS):

- Performance measure: reach the survivor at minimum total cost, safely
- Environment: the grid building with obstacles and risky zones

- Actuators: movement in 4 directions
- Sensors: adjacent-cell sensing only

Rationality: at every step (e.g., Step 1, Step 8), the robot picks the lowest-cost move available given only what it currently knows — this is bounded rationality, not perfect rationality, since it cannot see the dead end at (1,2) coming in advance. It only behaves optimally with respect to its current percepts, and corrects course once new information (the dead end) arrives.

Environment type:

- Partially observable — confirmed directly by Step 4, where the dead end could only be discovered by physically arriving there
- Deterministic — each move has a predictable result
- Sequential — Step 8's decision depends entirely on what was learned in Steps 4–7
- Discrete — grid cells, moves, and costs are all discrete
- Single-agent, static physical layout (only the robot's knowledge changes over time)

(d): Justification

An online search agent using cost-based (Uniform-Cost-style) move selection is the right approach here specifically because Step 4 in the trace above could not have been predicted by any offline algorithm — the robot only knows (1,2) is a dead end after sensing it, which is the core reality of a partially observable environment. A plain BFS/DFS would only count steps and could easily walk straight into the risky zone thinking it's equivalent to a normal cell; only cost-aware selection (Steps 1, 2, 8 above) correctly distinguishes a 1-cost move from a 3-cost move. And the backtracking in Steps 5–7 is exactly the behaviour a real physical rescue robot would need to perform — it cannot "undo" a move for free the way a simulated search tree can prune a branch instantly. This makes the online, cost-tracking approach the most realistic and correctly-behaved method for this exact problem.